
COMPUTING $F_{3 \times 10^{11}}$: ALGORITHMS, IMPLEMENTATION, AND PRACTICAL LIMITS

• Nathan Rousseau rousseaunathan84@gmail.com

May 24, 2026

ABSTRACT

We present the computation of the Fibonacci number $F_{3 \times 10^{11}}$, an integer containing more than 62 billion decimal digits. The computation relies on the fast doubling algorithm, whose correctness is proved in this work. The implementation (in C) uses the GNU Multiple Precision Arithmetic Library (GMP). We discuss practical challenges, including memory management and the verification of the result. The implementation is released as open-source software and is available at <https://github.com/STYC42/giant-fibonacci>.

Keywords Fibonacci numbers · Big integers · Algorithm analysis · Arbitrary-precision arithmetic

1 Introduction

The Fibonacci sequence, defined by $F_0 = 0$, $F_1 = 1$, and $F_{n+2} = F_{n+1} + F_n$ for all $n \in \mathbb{N}$, is one of the most studied integer sequences in mathematics. Despite its elementary definition, computing F_n for very large indices raises non-trivial algorithmic and engineering challenges that go well beyond asymptotic complexity analysis.

Motivation. Large Fibonacci computations arise naturally as stress tests for arbitrary-precision arithmetic libraries, as benchmarks for out-of-core algorithms, and as concrete illustrations of the gap between theoretical complexity and practical feasibility on commodity hardware. Unlike record computations of π [16] or large primes [7], which benefit from dedicated infrastructure and sustained community effort, Fibonacci numbers at the scale of tens of gigabytes have received comparatively little attention in the published literature. Research on large Fibonacci numbers has focused primarily on primality questions — the largest known Fibonacci prime is $F_{201,107}$, with 42,029 digits [10] — rather than on the computation of arbitrary Fibonacci values at extreme scale. One concrete illustration of this gap is the transition between computational regimes: Our measurements show that the algorithm behaves as a pure CPU-bound computation up to $n \sim 3 \times 10^{10}$, tracking the theoretical complexity $\Theta(n \log n \log \log n)$ to within 10%, before entering a disk-bound regime driven by SATA I/O throughput above that threshold. All raw timing logs are publicly available in the companion repository [13].

Contribution. In this work, we present the computation of $F_{3 \times 10^{11}}$, an integer containing approximately 6.27×10^{10} decimal digits (≈ 26 GB in binary representation). To our knowledge, no prior documented computation of a Fibonacci number at comparable scale exists in the literature.¹ The computation was carried out on a single workstation equipped with 64 GB of RAM and a SATA SSD, without distributed computing or specialised hardware.

The specific contributions of this paper are:

1. A presentation of the fast doubling recurrence (Theorem 1), which underlies the computation.
2. A description of an out-of-core multiplication algorithm (Algorithm 2) based on a classic-variant Karatsuba decomposition, with proofs of correctness, termination, and memory invariant.

¹We make no claim of an absolute record; undocumented computations may exist. Our claim is that this appears to be the largest *documented* computation of an arbitrary Fibonacci number to date, in the sense that both the result and the full computational trace are publicly archived and independently verifiable.

3. A characterisation of the practical bottlenecks encountered at this scale, including the GMP `mpz_t` size limitation, the memory requirements of the Schönhage–Strassen algorithm, and the I/O throughput constraints imposed by SATA storage. These are supported by a series of instrumented measurements spanning five orders of magnitude in n , which empirically locate the CPU-to-I/O transition between $n = 3 \times 10^{10}$ and $n = 3 \times 10^{11}$.
4. A modular verification procedure based on the Chinese Remainder Theorem, providing overwhelming probabilistic confidence in the correctness of the result. The verification completes in approximately 72 s, a cost dominated by loading the 26 GB result from disk rather than by the modular arithmetic itself.
5. A complete, self-contained C implementation of the above, released as open-source software at <https://github.com/STYC42/giant-fibonacci>, including the instrumented binary used to produce all measurements reported in this work.

Scope and limitations. This work does not aim to establish an optimised or minimal-resource implementation. The algorithm described is designed to be correct and analysable within the constraints of the available hardware, not to minimise wall-clock time. Extensions to larger indices — for instance $F_{6 \times 10^{11}}$ — would require either deeper recursive decomposition, distributed storage, or dedicated out-of-core multiplication routines beyond what is presented here.

Organisation. Section 2 presents the mathematical background, including the fast doubling formula and the asymptotic size of Fibonacci numbers. Section 3 describes the algorithms: fast doubling (Section 3.1), the out-of-core multiplication routine (Section 3.2), and their complexity analysis. Section 4 details the hardware environment and its implications for the computation. Section 5 presents the modular verification procedure. Section 6 concludes and discusses perspectives.

2 Mathematical Background

We present the mathematical background underlying the algorithms used in this work.

Theorem 1 (Fast Doubling). *For all $n \in \mathbb{N}$,*

$$F_{2n} = F_n \cdot (2F_{n+1} - F_n), \quad F_{2n+1} = F_n^2 + F_{n+1}^2$$

This identity is classical and appears, e.g., in [15, 3].

Proof. We proceed by induction on n .

Base case: For $n = 0$, we have $F_0 = 0$ and $F_1 = 1$, so

$$F_0 = 0 = F_0(2F_1 - F_0), \quad F_1 = 1 = F_0^2 + F_1^2.$$

Inductive step: Assume the formulas hold for some $n \in \mathbb{N}$, i.e.

$$F_{2n} = F_n(2F_{n+1} - F_n), \quad F_{2n+1} = F_n^2 + F_{n+1}^2.$$

We establish them for $n + 1$.

Even index. By the Fibonacci recurrence and the induction hypothesis,

$$\begin{aligned} F_{2(n+1)} &= F_{2n+1} + F_{2n} \\ &= (F_n^2 + F_{n+1}^2) + F_n(2F_{n+1} - F_n) \\ &= 2F_n F_{n+1} + F_{n+1}^2 \\ &= F_{n+1}(2F_n + F_{n+1}). \end{aligned}$$

Since $F_{n+2} = F_n + F_{n+1}$, we have $2F_n + F_{n+1} = 2F_{n+2} - F_{n+1}$, so

$$F_{2(n+1)} = F_{n+1}(2F_{n+2} - F_{n+1}).$$

Odd index. Using the result just established and the induction hypothesis,

$$\begin{aligned} F_{2(n+1)+1} &= F_{2n+2} + F_{2n+1} \\ &= F_{n+1}(2F_{n+2} - F_{n+1}) + (F_n^2 + F_{n+1}^2) \\ &= 2F_{n+1}F_{n+2} - F_{n+1}^2 + F_n^2 + F_{n+1}^2 \\ &= 2F_{n+1}F_{n+2} + F_n^2. \end{aligned}$$

Since $F_{n+2} = F_{n+1} + F_n$, we get

$$F_{n+2}^2 = (F_{n+1} + F_n)^2 = F_{n+1}^2 + 2F_{n+1}F_n + F_n^2,$$

hence $2F_{n+1}F_{n+2} + F_n^2 = 2F_{n+1}(F_{n+1} + F_n) + F_n^2 = F_{n+1}^2 + (F_{n+1} + F_n)^2 = F_{n+1}^2 + F_{n+2}^2$, so

$$F_{2(n+1)+1} = F_{n+1}^2 + F_{n+2}^2.$$

This completes the induction. $\square$

We need a formula for predicting the size of F_n . The intermediate results cannot be kept entirely in main memory, since the combined size of these integers largely exceeds the capacity offered by a standard computer.

Lemma 1. Let $\varphi = \frac{1+\sqrt{5}}{2}$ denote the golden ratio. Then

$$F_n \sim \frac{\varphi^n}{\sqrt{5}} \quad \text{as } n \rightarrow \infty.$$

Proof. The characteristic equation of the recurrence $F_{n+2} = F_{n+1} + F_n$ is $x^2 = x + 1$, with roots $\varphi = \frac{1+\sqrt{5}}{2}$ and $\psi = \frac{1-\sqrt{5}}{2}$. The general solution is $F_n = A\varphi^n + B\psi^n$; the initial conditions $F_0 = 0, F_1 = 1$ give $A = 1/\sqrt{5}$ and $B = -1/\sqrt{5}$, yielding Binet's formula [2, 11, 5]:

$$F_n = \frac{\varphi^n - \psi^n}{\sqrt{5}}.$$

Since $|\psi| = \frac{\sqrt{5}-1}{2} < 1$, we have $|\psi^n/\sqrt{5}| \rightarrow 0$, so $F_n \sim \varphi^n/\sqrt{5}$. $\square$

Theorem 2. The binary representation of F_n satisfies

$$\text{bits}(F_n) \sim n \log \varphi \quad \text{as } n \rightarrow \infty.$$

In particular, $\log \varphi \approx 0.6942$, so F_n has approximately $0.6942 n$ bits.

Proof. By Binet's formula, $F_n = (\varphi^n - \psi^n)/\sqrt{5}$. The number of bits required to represent a positive integer m is $\text{bits}(m) = \lfloor \log m \rfloor + 1$. Therefore

$$\text{bits}(F_n) = \left\lfloor \log \frac{\varphi^n - \psi^n}{\sqrt{5}} \right\rfloor + 1.$$

Write $\varphi^n/\sqrt{5} = \varphi^n/\sqrt{5}$ and factor:

$$\frac{\varphi^n - \psi^n}{\sqrt{5}} = \frac{\varphi^n}{\sqrt{5}} \left(1 - \left(\frac{\psi}{\varphi} \right)^n \right).$$

Since $|\psi/\varphi| = |\psi\varphi^{-1}| < 1$, the factor $(1 - (\psi/\varphi)^n) \rightarrow 1$, so

$$\log F_n = n \log \varphi - \frac{1}{2} \log 5 + \log \left(1 - \left(\frac{\psi}{\varphi} \right)^n \right).$$

The last two terms are $O(1)$, hence

$$\text{bits}(F_n) = n \log \varphi - \frac{1}{2} \log 5 + O(|\psi/\varphi|^n) + O(1) \sim n \log \varphi.$$

Numerically, $\log \varphi = \log\left(\frac{1+\sqrt{5}}{2}\right) \approx 0.69424$ [11, 5]. $\square$

For the calculation of $F_{3 \times 10^{11}}$, the doubling formula requires $F_{1.5 \times 10^{11}}$ and $F_{1.5 \times 10^{11}+1}$. A straightforward implementation would require all three values to be stored simultaneously in main memory, for a total size of

$$\begin{aligned} (3 \times 10^{11} + 2 \times 1.5 \times 10^{11} + 1) \cdot \log_2(\varphi) &\approx 0.69 \times 6 \times 10^{11} \\ &\approx 4.14 \times 10^{11} \text{ bits} \\ &\approx 51.8 \text{ GB.} \end{aligned}$$

In practice, large integer multiplication algorithms require substantial additional workspace. The actual memory consumption significantly exceeds the 52GB estimated before. This makes a purely in-memory computation unrealistic on standard hardware, motivating the use of external storage for intermediate results.

3 Algorithm

3.1 Fast doubling algorithm

The calculation of F_n relies on the fast doubling identity. From $F_0 = 0$ and $F_1 = 1$, the algorithm (see Algorithm 1) recursively computes (F_n, F_{n+1}) . Its depth is $O(\log n)$. The fast doubling method is described in [15, 3, 12].

Algorithm 1: Fast Doubling

```

Fib(n)
  if n = 0 then
    return (0, 1)
  end
  (a, b) ← Fib(⌊ $\frac{n}{2}$ ⌋);
  c ← a × (2 × b − a);
  d ← a2 + b2;
  if n mod 2 = 0 then
    return (c, d)
  end
  return (d, c + d)

```

For the calculation of $F_{3 \times 10^{11}}$, there will be 40 steps.

3.2 Out-of-core Multiplication

For integers of the size considered in this work, the high-level GMP type `mpz_t` becomes insufficient. For instance, the internal field `_mp_size`, which stores the number of limbs, is a signed 32-bit integer in standard GMP builds, while the size of a limb is 8 bytes. This limits the maximum number of limbs to $2^{31} - 1 \approx 2 \times 10^9$, whereas $F_{3 \times 10^{11}}$ requires approximately $\frac{26 \times 10^9}{8} \approx 3.7 \times 10^9$ limbs.

We therefore use GMP's low-level `mpn` interface, which allows direct control over size limitations while still benefiting from GMP's optimised arithmetic routines. The GMP library and its internal interfaces are documented in [6].

Let M denote a fixed memory threshold (set to approximately 4 GB on our machine, see Section 4). Each integer operand is represented as a structure carrying either an in-memory `mpn` limb array or a path to a temporary file on the SSD. The following primitives operate on such structures:

- **SPILL(x)**: if x is currently in memory, serialises its limb array to its associated temporary file and frees the in-memory allocation; if x is already on disk, this is a no-op.
- **RELOAD(x)**: if x is on disk, reads its temporary file back into a fresh limb array and marks x as in-memory; if x is already in memory, this is a no-op.
- **FREE(x)**: releases all resources held by x : the in-memory limb array if present, *and* the temporary file on disk if one exists. After this call, x is invalid.

We write $\text{mem}(t)$ for the total number of bits held in RAM at step t of the computation.

Let a and b be non-negative integers, and let $n = \lfloor (\text{bits}(a) + \text{bits}(b)) / 4 \rfloor$. Write

$$a = a_h \cdot 2^n + a_\ell, \quad b = b_h \cdot 2^n + b_\ell,$$

where $a_\ell = a \bmod 2^n$, $a_h = \lfloor a / 2^n \rfloor$, and similarly for b . The Karatsuba identity reads:

$$a \cdot b = z_2 \cdot 2^{2n} + ((a_h + a_\ell)(b_h + b_\ell) - z_2 - z_0) \cdot 2^n + z_0$$

where $z_0 = a_\ell \cdot b_\ell$, $z_2 = a_h \cdot b_h$, and $z_1 = (a_h + a_\ell)(b_h + b_\ell) - z_2 - z_0$. This decomposition is due to Karatsuba and Ofman [9].

Remark 1. The factors $(a_h + a_\ell)$ and $(b_h + b_\ell)$ are at most $n + 1$ bits each, which adds one bit of overhead compared to the signed variant. Since only one level of Karatsuba is applied in this work, this overhead is negligible in practice.

Algorithm 2: Out-of-core Multiplication

Input: Integers $a, b \geq 0$, both spilled to disk

Output: $s = a \times b$, spilled to disk

$n \leftarrow \lfloor (\text{bits}(a) + \text{bits}(b))/4 \rfloor$;

if $n < M$ **then**

 reload(a, b);

$s \leftarrow a \times b$;

 // GMP mpn_mul

 spill(a, b, s);

return s ;

else

 // -- Split phase --

 reload(a);

$(a_h, a_\ell) \leftarrow \text{split}(a, n)$;

 spill(a);

 reload(b);

$(b_h, b_\ell) \leftarrow \text{split}(b, n)$;

 spill(b);

 // -- Middle factor --

$s_a \leftarrow a_h + a_\ell$;

$s_b \leftarrow b_h + b_\ell$;

 spill($a_h, a_\ell, b_h, b_\ell, s_a, s_b$);

$z_1 \leftarrow \text{Mul}(s_a, s_b)$;

 free(s_a, s_b);

 // -- Corner products --

$z_0 \leftarrow \text{Mul}(a_\ell, b_\ell)$;

 free(a_ℓ, b_ℓ);

$z_2 \leftarrow \text{Mul}(a_h, b_h)$;

 free(a_h, b_h);

 // -- Combine --

 reload(z_0, z_1, z_2);

$s \leftarrow z_2 \cdot 2^{2n} + (z_1 - z_2 - z_0) \cdot 2^n + z_0$;

 free(z_0, z_1, z_2);

 spill(s);

return s ;

end

The implementation follows Algorithm 2 closely; see [13] for the full source.

Proposition 1 (Correctness of Algorithm 2). *For all integers $a, b \geq 0$, $\text{Mul}(a, b)$ returns $a \times b$.*

Proof. We proceed by case analysis.

Base case ($n < M$). The algorithm directly computes $a \times b$ via `mpn_mul` and returns the result. Correctness is immediate.

Recursive case ($n \geq M$). Let $a = a_h \cdot 2^n + a_\ell$ and $b = b_h \cdot 2^n + b_\ell$ as defined by the split. Set $z_0 = a_\ell b_\ell$, $z_2 = a_h b_h$, and $z_1 = (a_h + a_\ell)(b_h + b_\ell)$.

By the classical Karatsuba identity,

$$\begin{aligned} a \cdot b &= (a_h \cdot 2^n + a_\ell)(b_h \cdot 2^n + b_\ell) \\ &= z_2 \cdot 2^{2n} + (a_h b_\ell + a_\ell b_h) \cdot 2^n + z_0. \end{aligned}$$

Observe that

$$a_h b_\ell + a_\ell b_h = (a_h + a_\ell)(b_h + b_\ell) - a_h b_h - a_\ell b_\ell = z_1 - z_2 - z_0.$$

Substituting gives the combine expression in the algorithm, which equals $a \times b$. The recursive calls to `Mul` are correct by induction (the base case terminates since n strictly decreases at each level; see Proposition 2). $\square$

Proposition 2 (Termination). *Algorithm 2 terminates for all inputs.*

Proof. In the recursive branch, the three sub-calls to `Mul` operate on integers of bit-size at most $n + 1 < 2n \leq \text{bits}(a) + \text{bits}(b)$. The quantity $\text{bits}(a) + \text{bits}(b)$ is thus strictly decreasing along any branch of the recursion. Since it is a non-negative integer, the recursion must reach the base case $n < M$ after finitely many steps. $\square$

Proposition 3 (Memory bound). *At every point during the execution of `Mul`(a, b) with $n \geq M$, the total number of bits held in RAM satisfies $\text{mem}(t) \leq 8n$.*

Proof. We track $\text{mem}(t)$ at each labelled step of the recursive branch. All sizes are given in units of n bits; we write kn to mean “at most kn bits plus lower-order terms”.

Step	Operation	mem
Start	a, b spilled; RAM empty	0
After reload(a)	a in RAM	$2n$
After split(a)	a_h, a_ℓ, a in RAM	$4n$
After spill(a)	a_h, a_ℓ in RAM	$2n$
After reload(b)	a_h, a_ℓ, b in RAM	$4n$
After split(b)	$a_h, a_\ell, b_h, b_\ell, b$ in RAM	$6n$
After spill(b)	a_h, a_ℓ, b_h, b_ℓ in RAM	$4n$
Compute s_a, s_b	add s_a, s_b ($n + 1$ bits each)	$6n$
After spill($a_h, a_\ell, b_h, b_\ell, s_a, s_b$)	nothing	0
During <code>Mul</code> (s_a, s_b)	at most $8 \cdot \frac{n}{2} = 4n$ (induction)	$4n$
After z_1 spilled, s_a, s_b freed	nothing	0
During <code>Mul</code> (a_ℓ, b_ℓ)	at most $4n$ (induction)	$4n$
After z_0 spilled, a_ℓ, b_ℓ freed	nothing	0
During <code>Mul</code> (a_h, b_h)	at most $4n$ (induction)	$4n$
After z_2 spilled, a_h, b_h freed	nothing	0
After reload(z_0, z_1, z_2)	three results	$6n$
Combine step	intermediate sum	$8n$
After s spilled, intermediates freed	nothing	0

The maximum is $8n$, attained during the combine step. The inductive hypothesis for sub-calls holds since they operate on integers of size $n + 1 \approx n$, so the bound $8 \cdot (n/2) = 4n$ applies. The bound $\text{mem}(t) \leq 8n$ holds throughout. $\square$

Corollary 1. *On a machine with R bits of available RAM, Algorithm 2 can multiply two integers of combined bit-size $4n$ provided that $8n \leq R$, i.e. $n \leq R/8$. In particular, with $R = 64 \text{ GB} = 64 \times 8 \times 10^9$ bits, the algorithm handles inputs of combined size up to $4 \times R/8 = 32 \text{ GB}$. This is consistent with the present computation, where the first doubling step involves operands of combined size $\approx 26 \text{ GB} < 32 \text{ GB}$.*

Note that this bound assumes the base-case multiplication (`mpn_mul`) does not exceed the available RAM. In practice, GMP’s Schönhage–Strassen[14] routine requires additional workspace of approximately $3 \times$ the operand size (see Section 4), which tightens the effective limit to $n \leq R/12$.

3.3 Complexity

We analyse the asymptotic complexity of Algorithm 1, ignoring the memory-hierarchy effects treated in Sections 3.2 and 4.

Throughout this section we assume that:

1. integer multiplication of two b -bit operands costs $K \cdot b \log b \log \log b$ elementary operations, as achieved by the Schönhage–Strassen algorithm [14, 11];
2. integer addition of two b -bit operands costs $S \cdot b$ elementary operations;
3. $\text{bits}(F_n) \approx \text{bits}(F_{n+1})$, which is justified by $F_{n+1}/F_n \rightarrow \varphi$ and the fact that $\log \varphi < 1$, so consecutive Fibonacci numbers differ in bit-length by at most 1.

Cost of one doubling step. A doubling step transforms (F_n, F_{n+1}) into either (F_{2n}, F_{2n+1}) (even branch) or (F_{2n+1}, F_{2n+2}) (odd branch).

Both branches perform three multiplications and at most four additions. Writing $\beta_n := \text{bits}(F_n) \sim n \log \varphi$ (Theorem 2), the dominant cost is

$$C(n) = 3K \beta_n \log \beta_n \log \log \beta_n + O(S \beta_n). \quad (1)$$

The odd branch adds one extra addition $F_{2n} + F_{2n+1}$, contributing $S \beta_{2n} = O(S n)$, which is absorbed by the $O(S \beta_n)$ term. Hence $C(n)$ is the same, up to the implicit constant, for both branches.

Substituting $\beta_n \sim n \log \varphi$:

$$C(n) \sim 3K(\log \varphi) n (\log n + \log \log \varphi) (\log \log n + \log \log \log \varphi) \sim 3K(\log \varphi) n \log n \log \log n, \quad (2)$$

where the second equivalence holds because $\log \log \varphi$ and $\log \log \log \varphi$ are $O(1)$ constants.

Total cost. Algorithm 1 processes the binary representation of n from most significant to least significant bit, performing one doubling step per bit. At recursion depth $i \in \{0, \dots, L\}$ with $L := \lfloor \log n \rfloor$, the index being doubled is $n_i := \lfloor n/2^i \rfloor$, so the total cost is

$$T(2n) = \sum_{i=0}^L C(n_i). \quad (3)$$

Using (2) and the bound $n_i \leq n/2^i$:

$$T(2n) \lesssim 3K(\log \varphi) \sum_{i=0}^L \frac{n}{2^i} (\log n - i) \log(\log n - i). \quad (4)$$

Set $j = L - i$ and $\ell = \log n$; then $i = L - j$, $2^i = 2^L/2^j$, and $\log n - i \approx \ell - (L - j) \approx j$ for $L \approx \ell$. Substituting and using $2^L \sim n$:

$$\sum_{i=0}^L \frac{n}{2^i} (\log n - i) \log(\log n - i) = \sum_{j=0}^L \frac{n}{2^{L-j}} j \log j \sim \sum_{j=0}^L 2^j j \log j. \quad (5)$$

The last sum is dominated by its largest term ($j = L$):

$$\sum_{j=0}^L 2^j j \log j = 2^L L \log L \left(1 + \sum_{k=1}^L \frac{(L-k) \log(L-k)}{2^k L \log L} \right). \quad (6)$$

For fixed $k \geq 1$ and $L \rightarrow \infty$,

$$\frac{(L-k) \log(L-k)}{L \log L} = \left(1 - \frac{k}{L} \right) \left(1 + \frac{\log(1 - \frac{k}{L})}{\log L} \right) = 1 + O\left(\frac{k}{L}\right),$$

so

$$\sum_{k=1}^L \frac{(L-k) \log(L-k)}{2^k L \log L} = \sum_{k=1}^L 2^{-k} + O\left(\frac{1}{L}\right) = 1 + O\left(\frac{1}{L}\right). \quad (7)$$

Substituting (7) into (6), then using $2^L \sim n$ and $L = \lfloor \log n \rfloor \sim \log n$:

$$\sum_{j=0}^L 2^j j \log j \sim 2 \cdot 2^L L \log L \sim 2n \log n \log \log n. \quad (8)$$

Inserting (8) into (4):

$$T(2n) \sim 6K(\log \varphi) n \log n \log \log n. \quad (9)$$

Then

$$T(n) \sim 3K(\log \varphi) n \log n \log \log n \quad (10)$$

Theorem 3. The asymptotic complexity of Algorithm 1 is $T(n) = \Theta(n \log n \log \log n)$.

This matches the bound established in [15]. The linear terms in S contribute at most $O(Sn \log n)$, which is $o(T(n))$ and therefore does not affect the leading term.

3.4 I/O Complexity of the Out-of-Core Computation

The asymptotic analysis of Section 3.3 ignores memory-hierarchy effects. We now quantify the I/O cost — measured in bits transferred between RAM and disk — introduced by the out-of-core Karatsuba path (Algorithm 2).

Setup and notation. Write M for the spill threshold (Section 4). At recursion depth $i \in \{0, \dots, L\}$ of Algorithm 1, the doubling step operates on operands of combined bit-size

$$B_i := \text{bits}(F_{n_i}) + \text{bits}(F_{n_{i+1}}) \sim 2\beta_{n_i} \sim \frac{2\beta_n}{2^i}.$$

Algorithm 2 applies one level of Karatsuba to a multiplication $\text{Mul}(a, b)$ whenever the split parameter $\tilde{n} := \lfloor (\text{bits}(a) + \text{bits}(b))/4 \rfloor$ satisfies $\tilde{n} \geq M$. This condition is equivalent to $\text{bits}(a) + \text{bits}(b) \geq 4M$ i.e. $2\beta_n \geq 4M$. Define

$$d := \left\lceil \log \left(\frac{\beta_n}{2M} \right) \right\rceil, \quad (11)$$

so that the out-of-core path is activated at the d outermost doubling steps (depths $i = 0, \dots, d-1$) and the GMP direct path handles all deeper steps.

Remark 2. For the present computation, $2n = 3 \times 10^{11}$ so $\beta_n \approx 1.04 \times 10^{11}$ bits and $M \approx 3.2 \times 10^{10}$ bits (Section 4), giving $d = \lceil \log(1.04 \times 10^{11} / 6.4 \times 10^{10}) \rceil = 1$. All results below are stated for general d ; but the single-level case $d = 1$ recovers the concrete figures reported in Table 1.

I/O cost of a single out-of-core multiplication. Consider a call $\text{Mul}(a, b)$ with $\text{bits}(a) + \text{bits}(b) = B$ and split parameter $\tilde{n} = \lfloor B/4 \rfloor \geq M$. Tracing Algorithm 2, the explicit spill and reload operations on objects larger than M bits are:

Operation	Object(s)	Volume (bits)	Comment
RELOAD	a before split	$\sim B/2$	
SPILL	a after split	~ 0	a has not changed
RELOAD	b before split	$\sim B/2$	
SPILL	b after split	~ 0	b has not changed
SPILL	$a_h, a_\ell, b_h, b_\ell, s_a, s_b$	$\sim 3B/2$	
RELOAD	z_0, z_1, z_2 at combine	$\sim 3B/2$	
SPILL	result s	$\sim B$	
Total		$\sim 4B$	

Note that write and read operations are in the same amount. Objects smaller than M bits remain in RAM throughout (they fit by definition of the threshold) and are not counted. Hence, one call $\text{Mul}(a, b)$ with combined operand size B transfers

$$Q_{\text{mul}}(B) \sim 4B \text{ bits.} \quad (12)$$

I/O cost per doubling step. At depth $i < d$, each doubling step invokes *three* multiplications on operands of combined size $\sim B_i/2$ each (the three Karatsuba sub-products z_0, z_1, z_2 have operands roughly half the size of the original pair). The doubling step also spills and reloads its own inputs F_{n_i} and $F_{n_{i+1}}$, contributing $\sim B_i$ additional bits. The total I/O at depth i is therefore

$$Q_i \sim 3 \cdot Q_{\text{mul}}(B_i/2) + B_i \sim 3 \cdot 4 \cdot \frac{B_i}{2} + B_i = 7B_i \sim \frac{14\beta_n}{2^i} \quad (i < d). \quad (13)$$

Steps at depth $i \geq d$ operate entirely in RAM and contribute $Q_i = 0$.

Total I/O volume. Summing over the d out-of-core levels:

$$Q(n) = \sum_{i=0}^{d-1} Q_i \sim 14\beta_n \sum_{i=0}^{d-1} 2^{-i} = 14\beta_n \cdot \frac{1 - 2^{-d}}{1 - 1/2} < 28\beta_n. \quad (14)$$

The geometric series converges rapidly: already for $d = 1$, $Q(n) \sim 14\beta_n$; the depth-0 step alone accounts for more than half the total I/O budget regardless of d . In particular:

Corollary 2. *The total I/O volume of Algorithm 1 combined with Algorithm 2 is $Q(n) = \Theta(\beta_n) = \Theta(n)$ bits, dominated by the outermost Karatsuba level.*

Remark 3 (Consistency with measurements). *For $2n = 3 \times 10^{11}$, equation (13) at $i = 0$ predicts $Q_0 \sim 14 \beta_n \approx 14 \times 1.04 \times 10^{11} \approx 1.5 \times 10^{12}$ bits ≈ 182 GB. At the measured write throughput of 209 MiB/s and read throughput of 364 MiB/s (Section 4), this yields a predicted I/O time of*

$$t_{\text{I/O}} \approx \frac{Q_0/2}{209 \text{ MiB/s}} + \frac{Q_0/2}{364 \text{ MiB/s}} \approx 651 \text{ s},$$

where the 1/2–1/2 split reflects repartition in Table 3.4.

We now confront this prediction with experimental measurements.

Experimental setup. We measured wall-clock time for $n \in \{10^7, 10^9, 10^{10}, 3 \times 10^{10}, 3 \times 10^{11}\}$ on the hardware described in Section 4, using the instrumented binary (`make trace`). For each run we record the total doubling time and, separately, the cumulative time spent in `spill` and `reload` operations on transfers larger than 100 MB, which we take as a proxy for disk-bound I/O cost.

n	T_{obs} (s)	T_{pred} (s)	$T_{\text{obs}}/T_{\text{pred}}$	$T_{\text{I/O}}$ (s)	Karatsuba recursions
10^7	0.1	0.1	1.0	—	0
10^9	14.3	14	1.0	—	0
10^{10}	167.0	160	1.0	6.8	0
3×10^{10}	594.7	500	1.2	28.2	0
10^{11}	2312.3	1800	1.3	119.6	0
1.5×10^{11}	3540.6	2750	1.3	369.3	0
2×10^{11}	5488.0	3700	1.5	870.1	1
3×10^{11}	9701.7	5700	1.7	1990.0	1

Table 1: Measured doubling time T_{obs} , theoretical prediction $T_{\text{pred}} = K \cdot n \log n \log \log n$ (normalised on $n = 10^7$), and measured I/O time $T_{\text{I/O}}$ (disk transfers ≥ 100 MB only). Dashes indicate that I/O was not separately measurable at that scale (either fully cached or not yet instrumented at the Karatsuba level).

Results. Figure 1 plots these values on logarithmic axes. Three regimes are visible. For $n \leq 10^{10}$, the observed times track the theoretical curve closely (ratio ≤ 1.1): the computation is CPU-bound, the disk is served entirely from the Linux page cache, and the I/O component is near from zero. At $n = 3 \times 10^{10}$, the ratio rises to 1.2; the operands at the outermost doubling step reach 2.6 GB, approaching the GMP direct-multiplication threshold $M = 4$ GB (Section 4), and Schönhage–Strassen begins to stress main-memory bandwidth. Between $n = 10^{10}$ and $n = 3 \times 10^{11}$, the ratio reaches largely exceed 1 and continues to grow, reflecting limits of main memory bandwidth, the activation of the out-of-core Karatsuba path or the onset of true SATA-limited I/O.

Implementation and reproducibility. The complete source code of the implementation described in this work is publicly available at

<https://github.com/STYC42/giant-fibonacci>

The repository contains the main computation binary (written in C, targeting GMP’s `mpn` low-level interface), the modular verification program, and a `Makefile` providing the `trace` target used to produce the measurements reported in this work. Build instructions and the exact compiler flags used for the benchmarks are documented in `README.md`.

4 Hardware Environment

All computations were performed on a single workstation whose configuration is summarised in Table 2.

Justification of the threshold M

The threshold M controls the boundary between the standard GMP `mpn_mul` path and the out-of-core Karatsuba path. Its value is determined by two constraints.

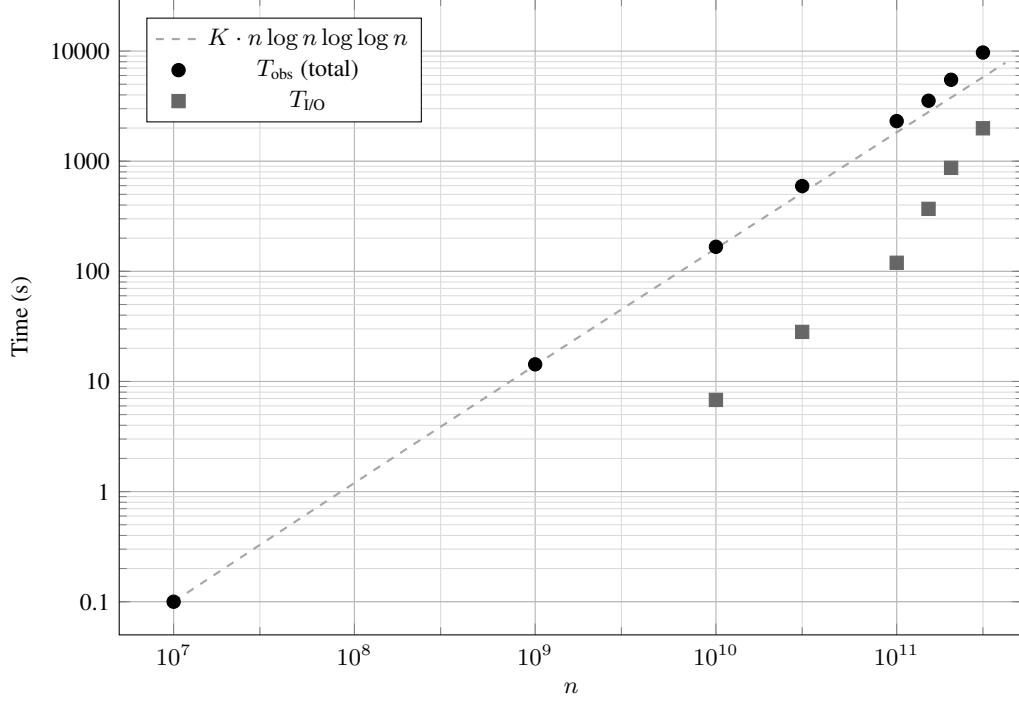


Figure 1: Wall-clock time versus n on double-logarithmic axes. Filled circles: total doubling time. Filled squares: disk I/O time (transfers ≥ 100 MB). Dashed line: theoretical prediction $T(n) \propto n \log n \log \log n$, normalised to the $n = 10^7$ measurement. The divergence between observed and predicted time above $n \sim 3 \times 10^{10}$ marks the onset of the out-of-core regime.

Component	Specification
CPU	Intel Xeon E5-1650 v4 @ 3.60 GHz (6 cores / 12 threads, Broadwell-EP)
RAM	64 GB DDR4 ECC (4×16 GB Samsung, 2400 MT/s)
Primary SSD	Crucial MX300 525 GB (SATA III) — temporary spill/reload files
Secondary HDD	Seagate Barracuda 2 TB (SATA III, 7200 rpm) — persistent storage
OS	Debian GNU/Linux (kernel 6.1.0-45-amd64, Bookworm)
GMP	version 6.2.1 (libgmp-dev 2:6.2.1+dfsg1-1.1, amd64)

Table 2: Hardware and software environment.

GMP internal multiplication strategy. GMP 6.2.1 transitions from Toom-Cook to the Schönhage–Strassen FFT-based algorithm at operand sizes of roughly 80 000–100 000 limbs (approximately 640 MB per operand on a 64-bit system)[6]. For two operands of combined size $4n$, the Schönhage–Strassen routine internally allocates working buffers of size proportional to the operand, with a practical overhead factor of approximately $3\times$. This means the total memory consumption of a single `mpn_mul` call can reach roughly $12n$ bits.

Derivation of M . With $R = 64$ GB of available RAM, we require

$$12n \leq R \implies n \leq \frac{R}{12} \approx 5.3 \text{ GB}.$$

Accounting for OS overhead, GMP internal bookkeeping, and the two operands themselves already occupying $4n$ bits, we set

$$M = 4 \text{ GB} \approx 3.2 \times 10^{10} \text{ bits}$$

as a conservative threshold that keeps peak in-memory usage within safe limits during the base multiplication step. This value was confirmed empirically: `mpn_mul` calls with combined operand size below $4 \cdot M = 16$ GB completed without triggering the Linux OOM killer[4], while calls above this threshold caused memory exhaustion.

Implication for the computation. The first doubling step — computing $F_{3 \times 10^{11}}$ from $F_{1.5 \times 10^{11}}$ — involves operands of combined size approximately $4n \approx 26$ GB, giving $n \approx 6.5 \text{ GB} > M$. A single level of the out-of-core Karatsuba

(Algorithm 2) is therefore applied, reducing each sub-multiplication to operands of combined size ≈ 13 GB, i.e. $n \approx 3.25 \text{ GB} < M$, which falls within the GMP direct path. All deeper doubling steps involve smaller operands and use GMP directly.

I/O cost of spill and reload operations

Raw device throughput. Sequential throughput on the Crucial MX300 was measured using `fio` (1 MiB blocks, `psync` engine, queue depth 1)[1]:

- **Read (reload):** 364 MiB/s.
- **Write (spill):** 209 MiB/s, measured with `-fsync=1` to bypass the OS page cache and reflect true on-disk throughput. During the actual computation, main memory was saturated (≈ 60 GB in use), leaving no capacity for write buffering; spill operations were therefore effectively synchronous throughout.

Observed throughput during computation. In practice, the throughput measured during the computation differs sharply from the raw device figures above, depending on the size of the operands involved. For all doubling steps below the out-of-core threshold — that is, for all steps except the outermost Karatsuba level — the temporary files fit comfortably within the Linux page cache (operands reach at most ≈ 2.6 GB at $n = 3 \times 10^{10}$, well below the 64 GB RAM). In this regime the instrumented traces report apparent write throughputs of ≈ 1900 MB/s and read throughputs of ≈ 3800 MB/s, reflecting RAM-to-RAM copies rather than true disk activity.

Remark on the secondary HDD. The Seagate Barracuda HDD is used solely for persistent storage of the final result. A side experiment running the computation with spill files redirected to the HDD at $n = 10^9$ produced no measurable difference in total time (ratio 1.007), confirming that at this scale the working set fits in the page cache regardless of the underlying device.

5 Validation of the Result

5.1 Verification strategy

For an integer bigger than 26GB, recomputing the value independently would require computational resources comparable to the original computation itself.

We therefore use modular verification techniques. The principle is simple:

- compute the large integer once,
- then verify its residues modulo many independent primes.

Since modular computations only manipulate machine-sized integers, verification is extremely fast while still providing overwhelming confidence in correctness.

5.2 Modular verification

Let X be the computed value claimed to equal $F_{3 \times 10^{11}}$.

We choose pairwise distinct prime numbers $p_1, p_2, \dots, p_k$ and independently compute $F_n \bmod p_i$ using a separate implementation restricted to fixed-size arithmetic.

X is then reduced modulo each p_i , and we verify for every i :

$$X \equiv F_n \pmod{p_i}$$

Suppose that the computation produced an incorrect value $X \neq F_n$. Define the error $D = X - F_n$.

If all modular checks succeed, then for every i , $D \equiv 0 \pmod{p_i}$.

Since the moduli are pairwise coprime:

$$D \equiv 0 \pmod{P}, \quad P = \prod_{i=1}^k p_i.$$

Thus any undetected error must satisfy $P \mid D$. In other words, the error must be an exact multiple of the product of all chosen primes.

The validity of the method relies on the Chinese Remainder Theorem.

Theorem 4 (Chinese Remainder Theorem). *Let $p_1, \dots, p_k$ be pairwise coprime integers and define $P = \prod_{i=1}^k p_i$. Then, for every tuple of residues $(a_1, \dots, a_k)$, there exists a unique integer modulo (P) satisfying:*

$$x \equiv a_i \pmod{p_i} \quad \forall i.$$

See, e.g., [8, 3].

Consequently, the collection of residues modulo the p_i uniquely determines an integer modulo P . Assume the computed value X passes all modular tests. Then X and $F_{3 \times 10^{11}}$ share exactly the same residue modulo:

$$P = \prod_{i=1}^k p_i.$$

Therefore $X \equiv F_n \pmod{P}$.

If $X \neq F_n$, then the difference must satisfy:

$$|X - F_n| \geq P.$$

Hence, increasing the number of moduli rapidly increases the minimum size of an undetected error.

If the error behaves essentially like a random integer, then the probability that it is divisible by a given prime p_i is $\frac{1}{p_i}$. Assuming independence between moduli, the probability that all checks succeed despite an incorrect result is approximately:

$$\prod_{i=1}^k \frac{1}{p_i} = \frac{1}{P}.$$

5.3 Practical verification

We use $k = 10$ pairwise distinct primes, all close to 2^{63} , listed in Table 3. Each prime p_i is a 63-bit prime, so $\log p_i \approx 63$ and the product of all moduli satisfies

$$P = \prod_{i=1}^{10} p_i \approx 2^{630}.$$

Table 3: The ten 63-bit primes used for modular verification. All satisfy $p_i < 2^{63}$, so that products $a \cdot b \bmod p_i$ fit in unsigned 64-bit arithmetic without overflow. The offset column gives $2^{63} - p_i$.

i	p_i	$2^{63} - p_i$
1	9223372036854775783	25
2	9223372036854775643	165
3	9223372036854774899	909
4	9223372036854774847	961
5	9223372036854774787	1021
6	9223372036854774629	1179
7	9223372036854774589	1219
8	9223372036854774523	1285
9	9223372036854774511	1297
10	9223372036854774421	1387

Probabilistic confidence. Suppose the computed value X differs from $F_{3 \times 10^{11}}$. Define the error $D = X - F_{3 \times 10^{11}} \neq 0$. If all k modular checks pass, then $p_i \mid D$ for every i , and since the p_i are pairwise coprime, $P \mid D$. Modelling D as a uniformly random non-zero integer, the probability that P divides D is at most $1/P \approx 2^{-630}$, which is negligible beyond any conceivable hardware error rate.

Deterministic interpretation. A fully deterministic guarantee would require $P > |D|$, i.e. $P > |X - F_{3 \times 10^{11}}|$. The size of $F_{3 \times 10^{11}}$ is approximately $2^{2.08 \times 10^{11}}$ bits (Theorem 2), whereas $P \approx 2^{630}$. Consequently, P is vastly smaller than the value being verified, and the deterministic criterion cannot be satisfied with a reasonable number of 63-bit primes: it would require

$$k \geq \left\lceil \frac{2.08 \times 10^{11}}{63} \right\rceil = 3,302,921$$

independent primes, making direct deterministic verification computationally impractical.

We therefore rely exclusively on the probabilistic interpretation. With $k = 10$ primes of size $\approx 2^{63}$, the probability of an undetected error is bounded above by $2^{-630} \approx 10^{-190}$, a confidence level that far exceeds the reliability of the underlying hardware.

Implementation independence. To guard against systematic software faults, the verification is implemented independently of the main computation:

- the main computation uses GMP low-level `mpn` arithmetic;
- the verification uses only native 64-bit `uint64_t` arithmetic, with no GMP dependency;
- the modular Fibonacci values $F_{3 \times 10^{11}} \bmod p_i$ are recomputed from scratch using the fast doubling recurrence with all intermediate results reduced modulo p_i at each step.

Since all arithmetic remains within 64-bit integers, the verification requires negligible memory. In practice, the dominant cost is loading the 26 GB result X from disk to compute its residues; this accounts for approximately 72 s of the total verification time at the sequential read throughput of 364 MiB/s (Section 4).

6 Conclusion

In this work, we presented the computation of the Fibonacci number $F_{3 \times 10^{11}}$, an integer containing more than 62 billion decimal digits.

The computation relies on the fast doubling recurrence, and while the asymptotic complexity of the algorithm is well known, practical computation at this scale raises major implementation challenges related to arbitrary-precision arithmetic and memory management.

The standard GMP high-level interface becomes insufficient for integers of this size because of internal size limitations and the memory requirements of large multiplication algorithms. To overcome these limitations, we used the low-level `mpn` interface together with an external-memory strategy and a manually controlled Karatsuba decomposition. This allowed the computation to remain within the constraints of a 64 GB machine.

We also developed a modular verification procedure based on the Chinese Remainder Theorem. By validating the result modulo several independent large primes using a separate implementation, we obtain overwhelming confidence in the correctness of the final value while keeping verification computationally inexpensive.

Beyond the specific computation of this integer, this work illustrates the gap between asymptotic algorithmic complexity and practical large-scale computation. Even algorithms with logarithmic recursion depth rapidly become constrained by memory hierarchy, disk throughput, and implementation details once operands reach tens of gigabytes.

The approach developed here could likely be extended further using distributed storage, deeper recursive multiplication schemes, or dedicated out-of-core multiplication algorithms. However, on commodity hardware, the present computation appears close to the practical limits achievable with current GMP-based techniques.

References

- [1] Jens Axboe. fio — Flexible I/O tester, version 3.35. <https://github.com/axboe/fio>, 2023. Used to measure raw sequential read/write throughput of the SATA SSD (`-bs=1MiB, -ioengine=psync, -iodepth=1, -fsync=1`).

- [2] Jacques Philippe Marie Binet. Mémoire sur l'intégration des équations linéaires aux différences finies à coefficients variables. *Comptes Rendus de l'Académie des Sciences*, vol. 17, pp. 559–567, 1843. First published derivation of the closed form now known as Binet's formula.
- [3] Richard Crandall and Carl Pomerance. *Prime Numbers: A Computational Perspective*. Springer, New York, 2005. Chapter 9 covers fast integer arithmetic including Karatsuba and Schönhage–Strassen; CRT is treated in Chapter 2.
- [4] Mel Gorman. *Understanding the Linux Virtual Memory Manager*. Prentice Hall, Upper Saddle River, NJ, 2004. Covers the OOM killer mechanism; see also https://www.kernel.org/doc/html/latest/mm/oom_kill.html.
- [5] Ronald L. Graham, Donald E. Knuth, and Oren Patashnik. *Concrete Mathematics: A Foundation for Computer Science*. Addison-Wesley, Reading, MA, 1994. Chapter 6: Special numbers, pp. 276–284.
- [6] Torbjörn Granlund and the GMP development team. GNU MP: The GNU multiple precision arithmetic library, version 6.2.1. <https://gmplib.org>, 2015. Full documentation of mpn low-level interface and internal multiplication thresholds.
- [7] Great Internet Mersenne Prime Search. Gimps discovers largest known prime number, 2024.
- [8] Godfrey Harold Hardy and Edward Maitland Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, Oxford, 2008. Chapter 3: The Chinese Remainder Theorem, revised by D. R. Heath-Brown and J. H. Silverman.
- [9] Anatolii A. Karatsuba and Yuri P. Ofman. Multiplication of multidigit numbers on automata. *Soviet Physics Doklady*, vol. 7, pp. 595–596, 1963. Original publication of the Karatsuba multiplication algorithm.
- [10] Maia Karpovich. f_{201107} is prime, 2023. Verified via elliptic curve primality proving; see <https://t5k.org/top20/page.php?id=39>.
- [11] Donald E. Knuth. *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*. Addison-Wesley, Reading, MA, 1997. Section 1.2.8: Fibonacci numbers, pp. 79–97.
- [12] Project Nayuki. Fast Fibonacci algorithms. <https://www.nayuki.io/page/fast-fibonacci-algorithms>, 2023. Accessible presentation of the fast doubling method with complexity analysis.
- [13] Nathan Rousseau. Fast-doubling fibonacci implementation in C with out-of-core Karatsuba multiplication. GitHub repository, <https://github.com/STYC42/giant-fibonacci>, 2026. Commit hash: 6a362b6. Accessed: May 24, 2026.
- [14] Arnold Schönhage and Volker Strassen. Schnelle Multiplikation großer Zahlen. *Computing*, vol. 7, no. 3–4, pp. 281–292, 1971. Original publication of the Schönhage–Strassen FFT-based integer multiplication algorithm.
- [15] Daisuke Takahashi. A fast algorithm for computing large Fibonacci numbers. *Information Processing Letters*, vol. 75, no. 6, pp. 243–246, 2000. Presents the fast doubling recurrence and its complexity analysis.
- [16] Alexander J. Yee. *y-cruncher: A multi-threaded pi-program*, 2021.